

Descrição: Execução prática das rotinas de teste planejadas, emprego de ferramentas para registro e validação dos resultados obtidos.

Pergunta: Se executarmos um sistema exaustivamente e não encontramos absolutamente nenhum erro, a nossa execução de testes foi um sucesso estrondoso ou o nosso roteiro de testes falhou em procurar nos lugares certos?

Execução de Teste Passo a Passo

A. Conceito:

Entender a execução de testes é como entender a diferença entre **ler uma receita de bolo e realmente ir para a cozinha**. Na teoria, tudo funciona; na prática, é onde descobrimos se o forno esquenta demais ou se faltou açúcar. Aqui está uma explicação dividida pelos pilares fundamentais desse conceito:

1. O que é, de fato, a Execução?

É um **processo dinâmico**. Enquanto o planejamento do teste é estático (documentos e ideias), a execução é o "mão na massa". É rodar o software com um objetivo claro: **procurar falhas**. Diferente do que muitos pensam, o objetivo do testador não é "provar que o sistema funciona", mas sim **tentar quebrá-lo** de forma estruturada. Se você encontrar um erro, o teste foi excelente, pois evitou que esse problema chegasse ao cliente final.

2. A Anatomia de um Caso de Teste

Para que o teste não seja apenas "clicar por clicar", usamos o **Caso de Teste**, que possui três elementos vitais:

- **Precondição:** O estado inicial necessário. (Ex: "O usuário deve estar logado e ter saldo na conta").
- **Dados de Entrada:** O que você vai digitar ou inserir. (Ex: "Valor da transferência: R\$ 50,00").
- **Resultado Esperado:** O que o software *deveria* fazer segundo a regra de negócio. (Ex: "O saldo deve diminuir R\$ 50,00 e o comprovante deve ser gerado").

3. O Fluxo Lógico de Execução

Para garantir que o teste seja científico e repetível, seguimos este roteiro:

1. **Preparação:** Conferir se o ambiente e as precondições estão prontos.
2. **Ação:** Executar os passos exatos descritos no roteiro.
3. **Uso do Oráculo:** Aqui entra a figura do "juiz". O **Oráculo** é qualquer fonte que nos diga qual é o comportamento correto (pode ser um documento de requisitos, um sistema antigo de comparação ou o conhecimento do especialista).
4. **Validação:** O Oráculo compara o que o sistema **fez** com o que ele **deveria ter feito**.

4. O "Mindset" do Testador

O texto traz uma quebra de paradigma importante:

- **Teste bem-sucedido:** Encontrou um erro novo. (Vitória! O erro foi capturado antes do usuário ver).
- **Teste mal-sucedido (do ponto de vista de busca):** Não encontrou nada. (O sistema parece estável, mas não houve "descoberta").

Essa transição da teoria para a prática exige **rigor**. Se você pular um passo ou ignorar uma pré-condição, o resultado do teste perde a validade técnica, pois você não saberá se o erro foi do sistema ou da forma como você testou.

B. Visualizando o Fluxo Lógico de Execução:



C. Exercícios Rápidos:

1. Escreva a pré-condição necessária para testar a funcionalidade "Excluir Produto" de um carrinho de compras.
2. Identifique o oráculo em um teste de transferência PIX.
3. Por que um teste que não acha defeitos não é considerado um teste de sucesso?
4. Cite um cenário real onde o uso de objetos "mock" (dados simulados) seria necessário.
5. Descreva 3 passos exatos para um caso de teste de recuperação de senha.

Emprego de Ferramenta de Documentação de Teste

A. Conceito:

Vamos explorar como as ferramentas de documentação transformam o caos de um projeto de software em uma estrutura organizada. Vamos seguir por esse conceito através de perguntas e exemplos práticos.

O Repositório Central: O "Cérebro" do Projeto

Imagine uma equipe de 20 pessoas testando um sistema bancário. Se cada um anotasse seus erros em planilhas individuais ou cadernos, como saberíamos o que já foi corrigido?

Ferramentas como **Jira** ou **TestLink** funcionam como um **Repositório Central**. Elas garantem:

- **Integridade:** Os dados não se perdem e não são alterados sem rastro.
- **Histórico:** Sabemos exatamente quem testou, quando testou e qual foi o erro encontrado.
- **Evidências:** Fotos e logs do sistema ficam guardados em um só lugar para provar que o teste foi feito.

A Rastreabilidade e a Matriz (RTM)

O foco técnico é a **Rastreabilidade**. Pense nisso como um fio que conecta tudo:

1. **O Requisito:** "O sistema deve aceitar pagamentos via PIX".
2. **O Caso de Teste:** "Tentar fazer um PIX de R\$ 10,00".

3. O Resultado: "Sucesso ou Falha".

A **Matriz de Rastreabilidade (RTM)** é a tabela que cruza essas informações. Se o cliente perguntar: "Vocês testaram a função de PIX?", você olha na matriz e mostra exatamente quais testes cobrem esse requisito. Isso permite calcular a **DRE (Eficiência na Remoção de Defeitos)**, que é basicamente uma conta matemática para saber se pegamos a maioria dos erros antes do lançamento.

B. Visualizando a Matriz de Rastreabilidade (RTM) Simplificada:



C. Exercícios Rápidos:

1. O que significa a sigla RTM e para que ela serve?
2. Como uma ferramenta de documentação previne o "caos" em um projeto grande?
3. O que é rastreabilidade reversa (retroativa)?
4. Quais são as três perguntas que um Relatório de Status (CSR) precisa responder?
5. Se o requisito REQ-02 não tem nenhum caso de teste associado na ferramenta, qual problema de qualidade temos?

Identificação de Problemas Sistêmicos e Registro de Bugs (Bug Report)

A. Conceito:

A identificação precisa de **problemas sistêmicos** exige que desenvolvamos nossa capacidade analítica para **distinguir tecnicamente** entre:

- **Erro** (ação humana que originou o problema),
- **Defeito** (o dado incorreto presente no código) e
- **Falha** (a manifestação funcional desse defeito durante a execução).

Quando detectamos a **falha**, entramos na **fase de registro do bug**. O registro é o **cadastro formal do defeito**, cujo objetivo absoluto é fornecer ao desenvolvedor todas as informações necessárias para a reprodução fiel e correção da falha encontrada. O registro eficiente minimiza o retrabalho e evita o descontrole da qualidade no projeto.

O foco central desta etapa é a **Reprodutibilidade**, pois a falha só será verdadeiramente corrigida se o programador conseguir **simulá-la exatamente** como ocorreu na máquina do testador. Um **Bug Report** técnico eficiente precisa de objetividade, passagens exatas para reprodução, taxonomia (como a categoria, severidade do impacto e prioridade de negócio), além de evidências visuais ou sistêmicas marcadas. O ciclo de vida desse defeito transitará por diversos estados, como **Aberto**, Em **andamento**, **Resolvido** e **Fechado**, garantindo governança sobre a qualidade.

Soft Skill importante: A redação do bug deve manter a **Neutralidade**, sendo baseada em fatos e sem carga emocional apontando dedos.

B. Visualizando o Ciclo de Vida do Defeito:



C. Exemplo de Código:

Markdown (exemplo.md)

Bug Report (Template Padrão SENAI)

****Título:**** Erro 500 ao enviar formulário com CEP vazio.

****Categoria:**** Interface/Validação | ****Severidade:**** Grave | ****Prioridade:**** Alta

****Precondição:**** Estar autenticado na tela de "Novo Endereço".

****Passos para Reprodução:****

1. Acessar a tela de cadastro de cliente.
2. Preencher os campos "Nome" e "Rua", deixando "CEP" em branco.
3. Clicar no botão "Salvar".

****Resultado Esperado:**** Sistema deve bloquear o envio e exibir "CEP obrigatório".

****Resultado Obtido:**** Sistema trava e exibe tela em branco com código HTTP 500.

****Evidências:**** [link-screenshot-log-console.png]

D. Explicação linha por linha do código:

- **Título:** Utiliza objetividade técnica sem ser vago ("o sistema quebrou").
- **Categoria/Severidade/Prioridade:** Aplica a Taxonomia do defeito para facilitar a triagem pela gerência do projeto.
- **Passos para Reprodução:** Oferece uma lista numerada exata, garantindo a reprodutibilidade vital para o desenvolvedor.
- **Esperado vs Obtido:** Confronto direto entre a especificação e a falha manifestada.
- **Evidências:** Anexo obrigatório (logs ou prints) para comprovar a falha tecnicamente.

E. Exercícios Rápidos:

1. Diferencie Erro, Defeito e Falha no contexto de desenvolvimento de software.
2. O que significa garantir a "Reprodutibilidade" em um report de bug?
3. Qual a diferença entre Severidade e Prioridade de um bug?

4. Reescreva de forma técnica o seguinte relato amador: "A tela de cadastro tá horrível e apagou tudo do nada!".
5. Por que um defeito pode ser devolvido ao status de "Feedback" pelo desenvolvedor?

Validação e Comparação de Resultados de Testes

A. Conceito:

Esta é a etapa crítica onde confrontamos o **Resultado Obtido** (o comportamento atual e visível do sistema) com o **Resultado Esperado** (nosso parâmetro de sucesso absoluto). É aqui que englobamos a **Verificação** ("Estamos criando o produto corretamente, seguindo as normas?") e a **Validação** ("Estamos criando o produto certo, que resolve a dor do usuário?"). O nosso objetivo principal nesta fase é a tomada de decisão concreta sobre o status do teste – **Aprovado ou Reprovado** – e garantir que o software construído satisfaça perfeitamente os critérios de aceite estabelecidos pelo cliente.

Como a realização de testes exaustivos cobrindo 100% dos cenários cósmicos é impossível, nossa validação **foca** de forma inteligente na conformidade com os **requisitos funcionais, comportamentais e lógicos visíveis**. A comparação técnica ocorre analisando saídas de texto, códigos HTTP de retorno e valores armazenados em banco. Se, durante um roteiro, o resultado obtido diferir do esperado, um defeito foi tecnicamente revelado (**Reprovado**). Se coincidirem em totalidade, o sistema está, até aquele momento, **Aprovado** na execução.

B. Visualizando a Matriz de Decisão de Teste:



C. Exemplo de Código:

JavaScript

// Exemplo de Verificação e Validação em um Teste Unitário (Jest)

```
const calcularFrete = require('./sistemaFrete');
```

```
test('Validação do Cálculo de Frete para Região Sul', () => {
```

```
  // Precondição e Massa de Dados
```

```
  const cepDestino = '89200-000'; // Joinville-SC
```

```
  const pesoKg = 5;
```

```
  const resultadoEsperado = 45.00; // Parâmetro de sucesso absoluto
```

```
  // Ação gerando o Resultado Obtido
```

```
  const resultadoObtido = calcularFrete(cepDestino, pesoKg);
```

```
// Validação e Comparação
expect(resultadoObtido).toBe(resultadoEsperado);
});
```

D. Explicação linha por linha do código:

- `const calcularFrete = ...`: Importa a função (módulo) que passará pela verificação.
- `const cepDestino ... const pesoKg`: Definem a massa de dados que operará sobre o domínio de entrada.
- `resultadoEsperado = 45.00`: Estabelece o parâmetro que dirá se nossa validação foi aprovada ou não.
- `resultadoObtido = calcularFrete(...)`: Captura a manifestação física do processamento lógico.
- `expect(resultadoObtido).toBe(resultadoEsperado)`: Motor de comparação. Se diferir, o teste falha revelando o erro; se bater, é aprovado.

E. Exercícios Rápidos:

1. Qual a diferença conceitual entre Verificação e Validação?
2. O que acontece imediatamente no projeto se o `Resultado Obtido` for diferente do `Resultado Esperado`?
3. O que são os Critérios de Aceitação?
4. Por que testes exaustivos são considerados inviáveis?
5. Dê um exemplo numérico simples (como em uma calculadora) que demonstre uma falha de validação.

FAQ - Dúvidas Reais

1. PERGUNTA: Achei um erro muito estranho, mas não consigo fazer ele acontecer de novo. Posso reportar mesmo assim?

RESPOSTA: É complicado. Se você não garante a **Reprodutibilidade** detalhando os passos exatos, o desenvolvedor provavelmente marcará o bug como "Rejeitado" por não conseguir simular. Tente usar ferramentas de gravação de tela para capturar a evidência no momento da execução.

2. PERGUNTA: Qual ferramenta do mercado nós devemos usar para documentar testes?

RESPOSTA: O mercado utiliza amplamente o *Jira* (com plugins como *Zephyr* ou *Xray*) e o *TestLink* (mais tradicional). O importante é o conceito por trás: manter o Repositório centralizado e a Rastreabilidade ativa.

3. PERGUNTA: Se o projeto está muito atrasado, podemos pular a documentação e só executar testes na interface?

RESPOSTA: Não é recomendado. Pular a base documentada impede a análise da DRE (Eficiência na Remoção de Defeitos) e quebra a Pirâmide de Teste, gerando retrabalho futuro alto e problemas não mapeados. Priorize riscos, mas mantenha o registro base.

4. PERGUNTA: O desenvolvedor corrigiu o defeito. O que eu faço no sistema de bugs?

RESPOSTA: O bug passa para o status de *Verificando (Homologação)*. Você reexecuta exatamente os mesmos passos para garantir a eficácia da correção. Só depois dessa revalidação o defeito transita para *Fechado*.

5. PERGUNTA: Como a IA Generativa (como o ChatGPT) pode ajudar o Engenheiro de Qualidade no dia a dia?

RESPOSTA: Pode ser utilizada para apoiar na criação de "Massas de Teste" ricas e complexas, gerando dados como CPFs válidos para testes ou estruturas JSON de entradas complexas, além de revisar textos de Bug Reports, garantindo a neutralidade técnica.

Atividade Prática: Transformação e Registro de Defeitos

A execução de testes é um processo dinâmico cujo objetivo não é provar que o software funciona, mas sim tentar quebrá-lo de forma estruturada para revelar falhas. Quando a validação falha — ou seja, o "Resultado Obtido" é diferente do "Resultado Esperado" — o teste é reprovado e exige a abertura imediata de um Bug Report.

Um Bug Report eficiente é a principal ferramenta de comunicação entre QA e Desenvolvimento. Ele exige reprodutibilidade exata, uso de taxonomia adequada (severidade e prioridade), e a aplicação da "Neutralidade" como *soft skill*, evitando apontamentos emocionais.

Abaixo, uma atividade prática avançada de **Registro Cirúrgico de Bugs**. O objetivo é que vocês é transformar os "relatos brutos" (Cenários) abaixo em relatórios formais utilizando o Template Padrão SENAI. Trabalhem em equipe, mas **a entrega é individual** no AVA.

Instruções da Atividade:

Para **cada um** dos 10 cenários brutos relatados abaixo, você deverá redigir **um Bug Report** formal contendo:

- Título (Objetivo e técnico)
- Categoria | Severidade | Prioridade (Taxonomia)
- Precondição (Estado inicial)
- Passos para Reprodução (Lista numerada exata)
- Resultado Esperado
- Resultado Obtido
- Evidências Sugeridas (Print, Log, etc.)

Utilizem editores de texto (Google Docs) para redigir seus Bug Reports.

Caso 1: O Colapso do PIX (Baseado em Requisitos e Integração)

- **Relato Bruto:** "Fui testar a transferência PIX de R\$ 50,00 que pediram. Entrei no app, botei a chave e cliquei em confirmar. O app fechou sozinho e não gerou comprovante nenhum."
- **Conexão com a Aula:** Falha no oráculo de transação. Alta severidade e alta prioridade.

Caso 2: A Falha Oculta na Regra de Negócio (Baseado na RTM)

- **Relato Bruto:** "O requisito R05 diz que pagamento no PIX tem 10% de desconto. Fui comprar um produto de R\$ 100,00, escolhi PIX e o carrinho cobrou R\$ 95,00."
- **Conexão com a Aula:** Falha de Validação de dados e regras de negócio baseada na Matriz de Rastreabilidade. O obtido (5% de desconto) diferiu do esperado (10%).

Caso 3: O Relato Amador e Emocional (Soft Skill)

- **Relato Bruto:** "O sistema burro travou tudo e apagou meu login de novo na tela de cadastro, resolve isso rápido pelo amor de Deus!!"
- **Conexão com a Aula:** Exige a aplicação de neutralidade técnica e objetividade para transformar um erro de perda de persistência de dados em um relato profissional.

Caso 4: A API que Devolve o Vazio (Baseado em Automação/Oráculos)

- Relato Bruto: "Estava testando o serviço de clima. Mandei a requisição esperando que a temperatura voltasse 25 graus, mas o servidor devolveu `{"status": 200, "temp": null}` e a tela do app ficou com 'NaN°C'."
- Conexão com a Aula: Confronto direto em testes de API e falha de validação lógica.

Caso 5: O Sucesso Invisível (Defeito de Interface)

- Relato Bruto: "Eu clico em 'Salvar' no meu perfil. As informações vão pro banco de dados certinho, mas não aparece a mensagem verdinha dizendo 'Salvo com sucesso' que o manual manda ter."
- Conexão com a Aula: Diferenciação entre defeito técnico (o banco salva) e falha de interface (requisito de UI não cumprido).

Caso 6: A Fronteira da Idade (Análise de Valor Limite)

- Relato Bruto: "A regra do sistema diz que só aceita idades de 18 a 65 anos. Botei 17 anos no campo de idade, cliquei em avançar e ele deixou criar a conta normalmente."
- Conexão com a Aula: Falha na massa de testes baseada em riscos e limites, permitindo violação de domínio.

Caso 7: A Precondição Ignorada (Estado Inicial)

- Relato Bruto: "Entrei no carrinho de compras vazio e a lixeira de 'Excluir Produto' estava ativa. Cliquei nela e deu Erro 500 na tela toda."
- Conexão com a Aula: A precondição para excluir um produto é *ter* um produto. O sistema falhou ao não tratar exceções de estado.

Caso 8: O Intermitente (Problema de Reprodutibilidade)

- Relato Bruto: "Às vezes, quando eu clico no botão de 'Curtir' na rede social, o coraçãozinho não fica vermelho. Acontece 1 vez a cada 10 tentativas. Não sei explicar direito."
- Conexão com a Aula: O maior desafio da triagem. Exigirá gravação de tela, logs do console e passos altamente detalhados para não ser devolvido com status de "Feedback".

Caso 9: O Loop da Senha (Fluxo Lógico Quebrado)

- Relato Bruto: "Pedi pra recuperar a senha. Chegou o e-mail. Eu clico no link do e-mail pra botar a senha nova, mas ele me joga de volta pra tela de 'Esqueci minha senha' de novo."
- Conexão com a Aula: O roteiro de 3 passos de recuperação de senha falhou na etapa de redirecionamento do Oráculo de segurança.

Caso 10: O Paradoxo Severidade vs. Prioridade

- Relato Bruto: "O sistema funciona inteiro, nada trava. Mas na página inicial, o nome da nossa empresa 'TechCorp' está escrito 'TecCrap' em letras garrafais vermelhas."
- Conexão com a Aula: Perfeito para o aluno exercitar que um bug de Severidade Baixa (não quebra funcionalidades) pode ter Prioridade Máxima (impacto absurdo de imagem e negócio).

Fechamento

Nesta aula, internalizamos como transformar o planejamento metódico em operação real. Esta execução rigorosa de rotinas e o registro cirúrgico de Bugs no fluxo de documentação serão o pilar central da nossa **Situação de Aprendizagem**. Nela, vocês atuarão como a equipe de QA auditando e validando as entregas de código uns dos outros, sendo avaliados pela qualidade dos reports e pela rastreabilidade que implementarem.

Na nossa próxima aula, subiremos mais um nível de maturidade e entenderemos a **Pirâmide de Testes e Estratégias de Automação Contínua (CI/CD)**. Entenderemos como robôs garantem a regressão dos defeitos que reportamos hoje, reduzindo custos e maximizando a segurança do software em entregas ágeis.